

CS3210 Assignment 3 Report

Darren Lim, Leong Yuhao

1 Implementation

We utilize a dynamic, flexible approach that **partitions both datapoints and queries across ranks** in a 2D mesh topology (grid). For example, datapoints may be split across rows, while queries may be split across columns, thus all threads would have their local datapoint and query share. Our algorithm can be broken down into the following components:

- **Dynamic Partitioning:** Based on the datapoint to query ratio, the master (rank 0) partitions by either datapoints or queries only, so the grid is 1 dimensional. **For the bonus**, it additionally computes the estimated runtimes across different grid configurations and selects the best one, consisting of the grid size, and the axes where the datapoints/queries are scattered. The configuration is then broadcast to all threads.
- **Data Transfer:** Within the grid, datapoints and queries are first scattered along their respective root axes, then broadcast along the other, such that all datapoint-query pairs are covered.
- **Main Computation:** Based on the number of attributes, either pairwise matching or a KD-tree is used for efficient searches.
- **Tree Reduction:** As each rank would only contain its local top- k nearest datapoints, all ranks transfer data back to the root query axis to obtain the global k nearest neighbours.
- **Send Back:** Ranks along the root query (scatter) axis transfer their results back to the master.

1.1 Program Flow

1.1.1 Data Transfer

Based on the estimated optimal configuration, datapoints are split by rows and queries by columns, or vice versa. Each rank is assigned two MPI communicators (`shared_data_comm` & `shared_query_comm`), based on their position in the grid, enabling communication among ranks within the same row or column. The master, first flattens datapoint and query information into contiguous arrays, while each rank allocates memory for incoming information. For simplification, we assume that datapoints are split by columns. `MPI_Scatterv` is used to partition datapoints equally and send them to each rank along row 0 (root ranks), before `MPI_Bcast` distributes the local datapoints from the root rank to all other ranks along each column. The same is carried out for queries, along the other axis.

1.1.2 Main Computation & Tree Reduction

To carry out the main distance computation, we use either pairwise matching or KD-trees. Queries are looped across in chunks of up to 64 (**Optim. 1**), where the local top- k nearest datapoints are computed for each query, stored as a pair of label and squared distance. To then obtain the global top- k , a manual reduction is implemented: one rank sends its top k datapoints to another, which finds the combined top- k . This is carried out recursively in a binary tree structure using `MPI_Send` and `MPI_Recv`. (**Optim. 2**)

- **Pairwise matching** checks each query against all datapoints in a thread, and stores the top- k candidates within a max-heap. This is used when `num_attrs` > 10.
- **KD-tree** is a binary tree that recursively splits datapoints into halves by checking one axis (i -th attribute) at a time, enabling logarithmic search times per query, but performance degrades in higher dimensions. Before processing the queries, all threads would first build a KD-tree from their local datapoints. This is used when `num_attrs` ≤ 10.

1.1.3 Send Back

At this step, the global KNN results would have been obtained, but still remains distributed across the root query axis. The master rank requests the remaining results using `MPI_Irecv`, thus avoiding the need to wait for sequential results. Immediately after, the master begins reporting the results, starting with its own share, while other ranks along the root query axis send their results back concurrently with `MPI_Send`.

1.2 Details

1.2.1 Data Structures

- **Flattened contiguous arrays** are used to transfer datapoint and query, and candidate/result information. This includes attributes (datapoints & queries), k (queries), distance and index (candidates & results). To benefit from cache-friendly memory accesses during pairwise matching, we do not reconstruct the flattened datapoints and queries as explicit structs, but use them as-is.
- **Max-heaps** of fixed size k are used to store the local candidates for each query. This allows each rank to efficiently maintain the current best results while scanning through its local query chunk.
- After local computation, the heap contents are converted into **sorted vectors** containing candidate (**distance**, **index**) pairs. These sorted vectors make merging more efficient during the reduction step, since two sorted lists can be directly combined, akin to mergesort, to produce the new top- k list.

1.2.2 Synchronization

`MPI_Bcast` is used at the start to ensure that all ranks receive the same global metadata, such as the number of datapoints, queries, and attributes, before any further work begins. During data distribution, `MPI_Scatterv` followed by `MPI_Bcast` synchronize the transfer of datapoint and query partitions across the grid, ensuring that every rank has the required local shard before computation starts.

During the reduction phase, synchronization is handled implicitly through matched `MPI_Send` and `MPI_Recv` calls. In each stage of the binary tree reduction, some ranks send their current local top- k candidate lists to partner ranks, while the other receiving ranks merge the results and continue to the next stage. This enforces a stage-by-stage dependency, since a rank cannot proceed until it has received and merged the required partial results from its partner.

In the final collection phase, rank 0 posts `MPI_Irecv` calls for the remaining query-group roots, allowing those incoming transfers to progress while rank 0 begins processing and reporting its own local results. Before using any received remote results, rank 0 calls `MPI_Wait` to ensure the corresponding non-blocking communication has completed. Finally, `MPI_Barrier` is used at the end of the program so that all ranks finish communication and computation before the communicators are freed and execution terminates.

2 Performance Data

2.1 Varying number of ranks (Implementation component)

For this experiment, we kept the parameters constant (see Appendix), and varied the number of ranks. We performed it on 2 of the same Intel i7-13700 nodes, and took the average of 5 runs.

As shown in the graph, the runtime of our program scales somewhat inversely with the number of ranks. As the number of ranks increases from 2 to 16, the average runtime across 5 runs decreases from 13386.4ms to 3689.6ms. (Refer to appendix for experimentation method)

This is because with more workers, each individual worker has less work to do, reducing the computation time and subsequently the overall runtime. As the number of ranks increases linearly, the performance increase (runtime decrease) also becomes less significant, suggesting a relationship corresponding to Amdahl’s Law, where the runtime plateaus towards the sequential portion, caused by inter-node communication and other overheads.

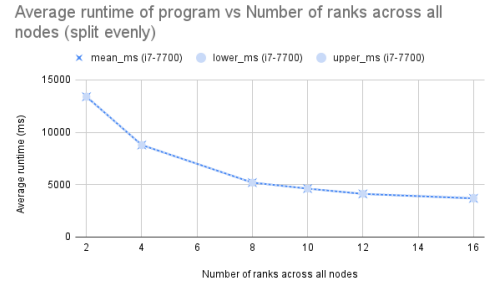


Figure 1: Runtime vs no. ranks

2.2 Varying number of datapoints

For this experiment, we kept the parameters constant (see Appendix), except for the number of datapoints (5k, 10k, 25k, 50k, 75k, 100k). We performed it on 2 of the same Intel i7-13700 nodes, and took the average of 5 runs. We note that the internal routine used throughout was pairwise matching.

As shown in the graph, the runtime increases approximately linearly with the number of datapoints. As number of datapoints increased from 5k to 100k, the average time taken increased from 707.2ms to 5739.4ms.

Expectedly, the total number of distance computations increases with datapoint size, since there are more datapoint-query pairs to check for. Moreover, the observed linear trend can be explained by computation being the dominant step. Thus, when other parameters are held constant, doubling the number of datapoints approximately doubles the total runtime.

2.3 Varying number of queries

Refer to appendix for experiment information. As shown in the graph, the runtime of the program increases as the number of queries is increased. From number of queries = 5000 to 100,000, the average runtime across 5 runs increased from 882.4ms to 14592ms. This is expected as the increase in the number of queries means that more computation has to be done to find the nearest neighbours for more queries.

2.4 Varying processors

For this experiment, we evaluated performance across 3 different processors (i7-7700, i7-13700, w5-3423). We fixed all parameters except for the number of ranks. The figure shows the average runtime for 2*i7-7700, 2*w5-3423, and 2*i7-13700. We split the tasks across the 2 processors equally.

According to the graph, the i7-13700 performed best, followed by the w5-3423 and finally the i7-7700 consistently across a different number of ranks. For number of tasks = 2, 2*i7-7700 took 13386.4ms, 2*w5-3423 took 11984ms, and 2*i7-13700 took 10249.2ms. For number of tasks = 16, 2*i7-7700 took 3689.6ms, 2*w5-3423 took 3404.4ms, 2*i7-13700 took 3243.8ms.

We attribute the difference in performance mainly to computation time. All processors benefit from a higher number of ranks used, as evidenced by the decrease in runtime. The performance of i7-13700 being the best can be attributed to it having the highest max turbo frequency of 5.20 GHz, as compared to w5-3423 (4.20 GHz) and i7-7700 (4.20 GHz). Moreover, the w5-3423 performed better than i7-7700 across varying number of tasks. This can be attributed to w5-3423 Cache size of 30MB vs 8MB for i7-7700. (Processor specifications cited in appendix)

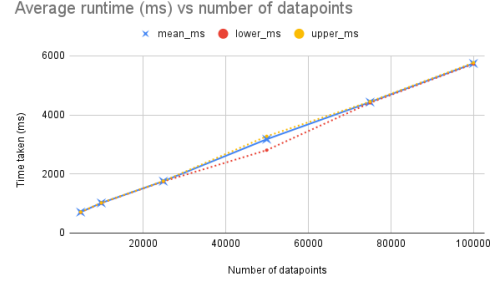


Figure 2: Runtime vs no. datapoints

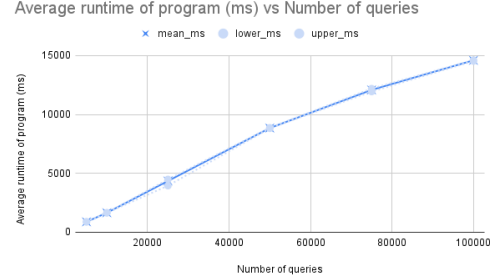


Figure 3: Runtime vs no. queries

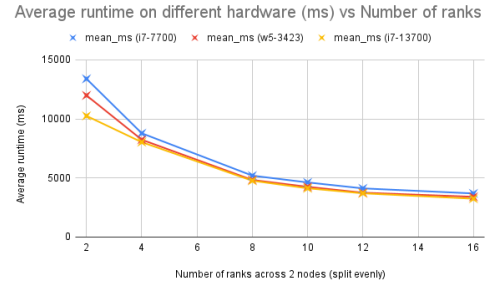
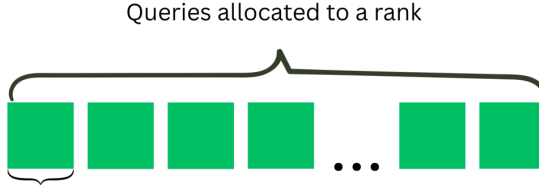


Figure 4: Runtime vs no. ranks, 3 hardware combinations

3 Optimization

3.1 Query Blocks



Process queries block by block:

Instead of computing the candidates for all local queries in a single pass, each rank computes the candidates for smaller query blocks, one by one. Prior to this optimization, each rank would have to iterate through all local queries per datapoint, which may constantly lead to cache replacements. Furthermore, more memory would have to be allocated for each heap corresponding to a query. This change keeps the loop cache-friendly by exploiting spatial and temporal locality.

As shown in the graph, the program with query blocks (optimized) performs better than without blocks (unoptimized), across different number of datapoints. The optimized version has a lower average runtime of 416ms (number of datapoints=5k) and 5505.2ms (number of datapoints=100k), as compared to the unoptimized version with 2698.2ms (number of datapoints=5k) and 21746.6ms (number of datapoints=100k).

3.2 Tree reduction

Inside each query block, the partial results are not sent back with a flat gather, but are merged recursively, in a binary tree fashion, over a `shared_query_comm`. Since ranks within a `shared_query_comm` share the same local queries but contain candidates computed from disjoint datapoints, there is a need to compute a global top- k . First, with step size = 1, pairs of consecutive ranks merge to obtain the combined top- k candidates. Subsequently, the step size is doubled, and the ranks containing the combined candidates merge again. This is repeated until rank 0 contains the global top- k at the end, allowing work to be shared across ranks.

Flat reduction means every rank sends its partial result directly to the root, where merging is performed and work is not shared. This is the unoptimized version.

As shown in the graph, the code with the tree reduction (optimized) performed better than flat reduction (unoptimized) across different number of datapoints. The optimized version has a lower average runtime of 416ms (number of datapoints=5k) and 5505.2ms (number of datapoints=100k), as compared to the unoptimized version with 419.2ms (number of datapoints=5k) and 6492ms (number of datapoints=100k).

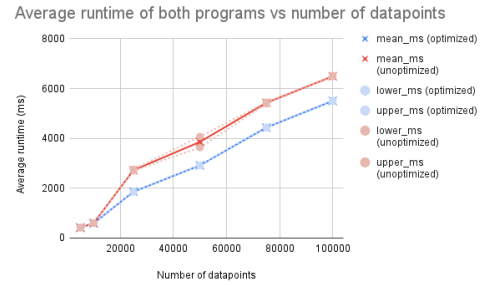
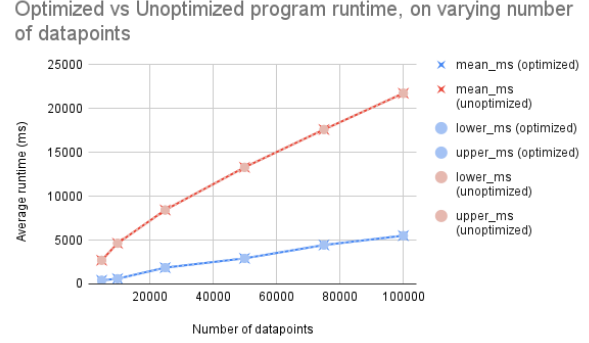


Figure 6: Runtime vs no. datapoints, optimized vs unoptimized

4 Bonus: Dynamic Partitioning

4.1 Motivation

While testing our initial program, we observed that communication times varied drastically across different node combinations. For example, across different combinations of 2*i7-13700s, those with `soctf-pdc-035/036` **consistently** took around 10× the communication time of those without! While communication usually accounted for a small fraction of the total runtime, this was not always the case.

Fortunately, since our test program had the ability to accept user-specified grid sizes, a potential source for optimization was to find the optimal configuration minimizing communication time for different inputs and node combinations.

4.2 Grey-Box Model

First, we had to understand how different grid sizes corresponded to different “Send Data(point)”, “Send Query”, “Reduce” and “Send-back” timings. A greater split did not always reduce the communication timings, and could even cause a time spike in some cases!

For example, when running on 2 nodes of the same processor type and scattering data of size N across rows, 2-ways vs 4-ways, both result in an inter-node transfer quantity of $N/2$, thus producing roughly equal scatter timings. Intra-node transfer times were also found to be magnitudes lower (≈ 0.01) than inter-node transfer times, and could be effectively ignored. We developed a grey-box model that could capture the relationship between different configurations, input sizes, and communication timings (equations in Appendix), by modelling the grid itself. This would then be utilized within the program.

4.3 NIC, the Unpredictable

However, we also discovered that partitioning information by columns caused certain communication timings to deviate from our grey-box model. We modelled the deviation as a multiplicative penalty applied to broadcasts and reductions along the in-contiguous axis, leading to the aforementioned time spikes.

Further investigation revealed that this penalty was likely a product of Network Interface Controller’s (NIC) internal algorithm for data transmission. While the penalty was found to be bounded above by the maximum number of worker threads among the non-master nodes, its behaviour was still largely inconsistent under different node configurations. This meant that training the grey-box to fit all node configurations would lead to significantly less accurate predictions as compared to training on fixed node configurations. Hence, we decided to utilize the grey-box only when the bonus nodes `soctf-pdc-[031,032]` were used, and defaulted to a 1D partitioning scheme for other nodes, to avoid paying the penalty for in-contiguous communication.

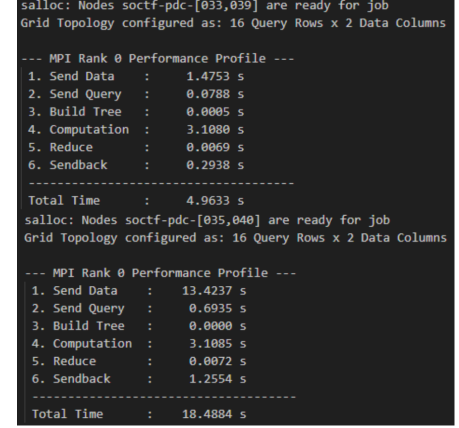


Figure 7: The culprit in action.

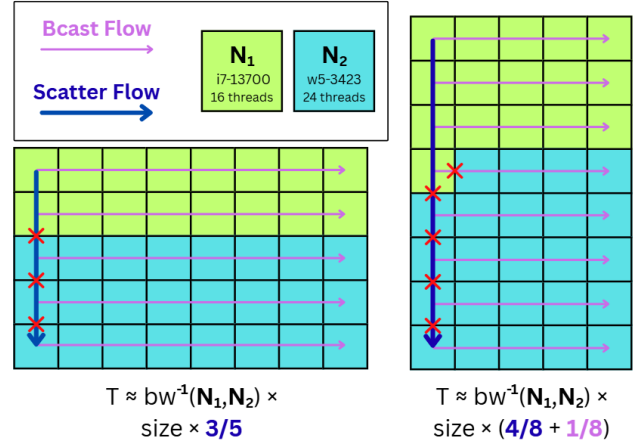


Figure 8: Estimate of scatter + broadcast time for 2 different grid configurations. Inverse bandwidth bw^{-1} is a parameter extracted between each pair of nodes.

Appendix A Methodology

Login Node: soctf-pdc-025.comp.nus.edu.sg, Intel(R) Xeon(R) w5-3423

Compiling: Run make with the given Makefile.

A.1 Varying number of ranks

Generated input:

```
python3 generate_input.py --num_data 25000 --num_queries 20000 --num_attrs 50 --min -100
--max 100 --minK 100 --maxK 100 --num_labels 10 --output inputs/vary_datapoints/25000.txt
```

We kept the parameters constant, except for number of tasks and combination of nodes.

We collected the data for number of tasks = 2, 4, 8, 10, 12, 16, which was split across 2 nodes equally by specifying `--ntasks-per-node`.

We collected this data on 2x Intel i7-7700 nodes, soctf-pdc-015 and soctf-pdc-016.

A.2 Varying number of datapoints

Generated inputs, where $X = \{5, 10, 25, 50, 75, 100\} \times 10^3$:

```
python3 generate_input.py --num_data X --num_queries 20000 --num_attrs 50 --min -100
--max 100 --minK 100 --maxK 100 --num_labels 10 --output inputs/vary_datapoints/5000.txt
```

We kept all the parameters constant except for the number of datapoints. We performed the experiment on the same 2x Intel i7-13700 nodes, soctf-pdc-033 and soctf-pdc-034, and took the average across 5 runs.

We modified the `run_bench.sh` and created a new script to automate this testing, and call it with:

```
./collect_vary_datapoints.sh
```

A.3 Varying number of queries

Generated inputs, where $X = \{5, 10, 25, 50, 75, 100\} \times 10^3$:

```
python3 generate_input.py --num_data 50000 --num_queries X --num_attrs 50 --min -100
--max 100 --minK 100 --maxK 100 --num_labels 10 --output inputs/vary_queries/5000.txt
```

We kept all the parameters constant except for the number of datapoints. We performed the experiment on the same 2x Intel i7-13700 nodes, soctf-pdc-033 and soctf-pdc-034, and took the average across 5 runs.

A.4 Varying processors

Generated input:

```
python3 generate_input.py --num_data 25000 --num_queries 20000 --num_attrs 50 --min -100
--max 100 --minK 100 --maxK 100 --num_labels 10 --output inputs/vary_datapoints/25000.txt
```

We kept the parameters constant, except for number of tasks and combination of nodes.

We collected the data for number of tasks = 2, 4, 8, 10, 12, 16, which was split across 2 nodes equally by specifying `--ntasks-per-node`.

We ran on 3 combinations, 2x Intel i7-13700 (soctf-pdc-037, soctf-pdc-038), 2x Intel w5-3423 (soctf-pdc-030, soctf-pdc-031), and 2x Intel i7-7700 (soctf-pdc-015, soctf-pdc-016). **None of these configurations was noted to suffer from poor bandwidth.**

We collected data by calling the script with the inputs:

```
./collect_data.sh --input inputs/vary_datapoints/25000.txt
--output outputs/vary_datapoints/25000.txt --nodes soctf-pdc-033,soctf-pdc-034
--ntasks-per-node 8 --runs 5
```

A.5 Optimization 1 & 2

We used the same input files as specified under varying number of datapoints.

We kept all the parameters constant except for the number of datapoints. We performed the experiment on the same Intel i7-13700 nodes, soctf-pdc-039 and soctf-pdc-040, and took the average across 5 runs.

We created a new script to automate this data collection and call it with:

```
bash automate_collect_data.sh
```

Appendix B Data

Link to Google Sheets:

- Experiment Data (main)
- Grey-Box Train Data

Appendix C Grey-Box Model

C.1 Equations

$$\begin{aligned}
(*) T_{scatter} &= w_s \cdot \underbrace{\frac{AN_d}{S_d}}_{\text{local datapoints}} \cdot \sum_{i=1}^{S_d-1} bw^{-1}(t_{0,0}, t_{i,0}), \\
(*) T_{broadcast} &= w_s \cdot \frac{AN_d}{S_d} \cdot \underbrace{\max_i \sum_{j=1}^{S_q-1} bw^{-1}(\text{unique node pairs of } t_{i,0}, t_{i,j})}_{\text{broadcast bottleneck path}}, \\
T_{reduce} &= \underbrace{\frac{KN_q}{S_q}}_{\text{local results}} \cdot \left(w_{r1} \cdot \underbrace{\max_i \sum_{s=1}^{\lceil \log_2 S_d \rceil} \left(\sum_{2^s \text{ Merge}} bw^{-1}(t_{src,i}, t_{dest,i}) \right)}_{\text{reduce bottleneck path}} + w_{r2} \cdot \underbrace{\lceil \log_2 S_d \rceil}_{\text{merge computation}} \right), \\
T_{sendback} &= w_{sb} \cdot \frac{KN_q}{S_q} \cdot \sum_{i=1}^{S_q-1} bw^{-1}(t_{0,0}, t_{0,i}),
\end{aligned}$$

where A is the number of attributes, K is the average neighbours, N_d, N_q are the number of datapoints and queries respectively, S_d, S_q are datapoint and query split respectively, $bw^{-1}(\cdot, \cdot)$ denotes the inverse bandwidth between the (nodes of) input ranks, and $t_{i,j}$ is the rank at row i , column j . The trainable parameters are $w_s, w_{r1}, w_{r2}, w_{sb}$, and the inverse bandwidth pair(s). (*) indicates that the equation can also be applied to queries, instead of datapoints, with the suitable adjustments.

C.2 Trainer

The trainer can be found in the file `hells_kitchen.ipynb`, located within the repository.

Appendix D References

Google Gemini was used to generate code for the following portions:

- Main program, `engine.cpp`, and variants, namely for KD-tree implementation, grid configuration.
- Grey-box model in Python.
- Experiment/batch generation scripts (`./create_manifest.sh`, `./run_experiment.sh`, `gen_master.cpp`)